
Projet d'Intelligence Artificielle : NoName

Lucas Fagioli

3 Mars 2026

Université de Lorraine UFR MIM
3 Rue Augustin Fresnel, 57070 Metz

Professeur : Alexandre Blansché

Résumé

Dans ce projet, j'ai développé en Java un bot capable de jouer à l'Awélé de manière autonome, en respectant les contraintes imposées par le tournoi : 100 ms maximum par décision, une heure d'apprentissage au démarrage, une mémoire limitée et l'interdiction de modifier le moteur fourni. Mon objectif a été d'obtenir un bot le plus performant possible malgré ces limites.

Pour cela, j'ai construit un moteur de recherche basé sur MinMax avec élagage Alpha-Beta, puis je l'ai progressivement optimisé. J'ai notamment implémenté une représentation bit à bit du plateau afin d'accélérer les copies d'états et la simulation des coups, ainsi qu'une table de transposition, un ordonnancement des coups et une recherche à profondeur adaptative (*Iterative Deepening*). J'ai également conçu une fonction d'évaluation heuristique combinant plusieurs critères, avec des poids différents selon la phase de jeu.

Enfin, pour éviter un réglage manuel des poids, j'ai mis en place une phase d'auto-jeu permettant de les optimiser automatiquement avec CMA-ES. Les expérimentations présentées dans ce rapport montrent l'impact des optimisations et l'amélioration des scores au cours de l'apprentissage, ce qui m'a permis d'aboutir à un bot que je considère satisfaisant et encore améliorable.

Table des matières

1	Introduction	1
2	Le jeu de l'Awélé	1
2.1	Présentation générale	1
2.2	Déroulement d'un tour	1
2.3	Règles particulières	2
2.4	Fin de partie	2
3	Analyse du problème et choix méthodologique	2
3.1	Modélisation du problème	2
3.2	Approches envisageables	2
3.3	Choix retenu	3
4	Méthodologie	3
4.1	Algorithme de recherche : MinMax	3
4.2	Représentation du plateau	3
4.3	Optimisation de la recherche	4
4.3.1	Élagage Alpha-Beta	4
4.3.2	Table de transposition	4
4.3.3	Ordonnancement des coups (Move Ordering)	5
4.3.4	Réduction sélective des coups (Late Move Reductions)	6
4.3.5	Profondeur adaptative (Iterative Deepening)	6
4.4	Fonction d'évaluation	6
4.5	Optimisation des paramètres	7
4.5.1	Algorithme génétique (GA)	7
4.5.2	Simultaneous Perturbation Stochastic Approximation (SPSA)	7
4.5.3	Covariance Matrix Adaptation Evolution Strategy (CMA-ES)	8
4.6	Apprentissage du bot	9
5	Résultats	10
5.1	Benchmark de la représentation du plateau	10
5.2	Temps de décision en fonction de la profondeur	10
5.3	Optimisation des poids par apprentissage	10
6	Conclusion	11
	Références	12
A	Pseudo-code de l'algorithme CMA-ES (Wikipédia)	13
B	Temps de décision selon la profondeur (mesures complètes)	14
C	Visualisation de l'évolution des poids	14
C.1	Évolution de la distribution CMA-ES	14
C.2	Évolution des poids (early / late)	16
D	Améliorations envisagées	17

1 Introduction

Dans le cadre de l'UE *Intelligence Artificielle*, il nous est demandé de développer en Java un bot capable de jouer au jeu de l'Awélé de manière autonome. Le projet s'appuie sur un moteur de jeu fourni.

Le développement du bot doit respecter plusieurs contraintes :

- **100 ms maximum** pour calculer un coup.
- **1 heure maximum** d'apprentissage au démarrage.
- **64 MiB maximum** de mémoire par bot.
- **1 GiB maximum** de mémoire totale pour le programme.
- **Moteur imposé** : modification du code existant interdite.

Ces contraintes obligent à concevoir un bot à la fois efficace, rapide et peu gourmand en mémoire.

Dans ce rapport, je présenterai tout d'abord une description du jeu et une analyse du problème du point de vue de l'intelligence artificielle. Je détaillerai ensuite la méthodologie adoptée pour concevoir le bot, notamment les choix algorithmiques et les optimisations mises en œuvre. Enfin, j'analyserai les résultats obtenus et discuterai des performances du modèle développé.

2 Le jeu de l'Awélé

2.1 Présentation générale

L'Awélé est un jeu africain traditionnel appartenant à la famille des *Mancala*¹ [1]. Il oppose deux joueurs autour d'un plateau composé de deux rangées de six trous, contenant initialement quatre graines chacun. Le principe du jeu repose sur la distribution successive des graines dans les trous du plateau, avec pour objectif d'en capturer un maximum.

Du point de vue théorique, l'Awélé appartient à la classe des jeux déterministes à information parfaite et à somme nulle. Déterministe signifie qu'aucun hasard n'intervient : un état du jeu et une action donnée conduisent toujours au même état suivant. L'information parfaite implique que les deux joueurs connaissent en permanence l'intégralité de la configuration du plateau ainsi que les scores respectifs. Enfin, le caractère à somme nulle traduit le fait que les intérêts des joueurs sont strictement opposés : maximiser son gain revient à minimiser celui de l'adversaire.

Ces propriétés permettent de modéliser le jeu sous la forme d'un arbre de recherche où chaque nœud représente un état du plateau et chaque arête un coup possible. L'objectif devient alors de déterminer, parmi l'ensemble des actions disponibles, celle qui optimise le résultat final en supposant que l'adversaire joue de manière optimale.

Avant ce projet, je ne connaissais pas ce jeu. Ses caractéristiques le rapprochent néanmoins d'un point de vue algorithmique d'un jeu que j'ai déjà pu étudier par le passé : les échecs. Cette similarité m'a servi de base de réflexion pour aborder la conception de mon bot.

2.2 Déroulement d'un tour

À chaque tour, le joueur :

- choisit un trou non vide de son camp ;
- récupère toutes les graines du trou sélectionné ;
- distribue les graines une à une dans les trous suivants (sens anti-horaire) ;
- ignore le trou d'origine en cas de tour complet ;
- applique, le cas échéant, la règle de capture.

1. Famille de jeux traditionnels fondés sur la distribution de graines dans des trous.

2.3 Règles particulières

Le jeu est encadré par les règles spécifiques suivantes :

- **Capture** : 2 ou 3 graines dans le camp adverse $\Rightarrow$ graines capturées.
- **Capture en chaîne** : prolongation tant que la condition reste vérifiée.
- **Famine** : interdiction de priver l'adversaire de graines (sauf impossibilité).
- **Trou vide** : un trou sans graine ne peut pas être joué.

2.4 Fin de partie

La partie se termine lorsqu'une des fonctions suivantes est vérifiée :

- un joueur atteint au moins 25 graines ;
- un joueur ne peut plus jouer de coup valide ;
- il ne reste plus suffisamment de graines pour permettre une capture significative.

Les graines restantes sont alors attribuées au joueur concerné et le vainqueur est celui qui possède le plus grand nombre de graines.

3 Analyse du problème et choix méthodologique

3.1 Modélisation du problème

L'Awélé peut être modélisé comme un ensemble d'états reliés par des actions. Un état correspond à une configuration complète du plateau (répartition des graines et scores), et une action correspond à un coup légal. La succession des coups forme ainsi un arbre de recherche où les joueurs alternent à chaque niveau.

L'espace d'états est très grand : il est estimé à 889 063 398 406 positions distinctes (près de 9×10^{11}). Cette taille rend toute exploration exhaustive irréaliste.

Le problème peut alors être formulé ainsi :

Comment déterminer le meilleur coup à jouer sans pouvoir explorer l'ensemble des positions possibles du jeu ?

Deux aspects deviennent alors centraux :

- **La profondeur de recherche** : explorer l'arbre le plus profondément possible afin d'anticiper les conséquences futures des décisions.
- **L'évaluation des nœuds** : attribuer une valeur pertinente aux positions non terminales lorsque la recherche doit être interrompue.

3.2 Approches envisageables

Plusieurs méthodes peuvent être envisagées pour résoudre ce problème :

- **Recherche exhaustive** : exploration complète de l'arbre de jeu jusqu'aux états terminaux (impraticable en raison de la taille de l'espace d'états).
- **Recherche MinMax avec élagage** : exploration des coups possibles sur plusieurs tours afin de trouver une issue favorable.
- **Méthodes Monte Carlo** : estimation de la valeur d'un coup par simulations aléatoires répétées à partir de l'état courant [2].
- **Apprentissage supervisé** : apprentissage d'une fonction d'évaluation à partir d'un ensemble de parties ou de positions annotées.
- **Apprentissage par renforcement** : amélioration progressive de la stratégie par auto-jeu et mise à jour des paramètres.

3.3 Choix retenu

Au vu des contraintes du projet (cf. section 1), une approche par apprentissage supervisé m'a paru difficile à mettre en œuvre de manière robuste.

J'ai donc choisi d'adopter une approche fondée sur l'algorithme MinMax, adapté à la nature déterministe du jeu.

Cependant, un MinMax naïf ne permettrait pas d'atteindre une profondeur suffisante. J'ai donc progressivement enrichi le modèle avec les éléments suivants :

- un algorithme MinMax avec élagage Alpha-Beta² ;
- une représentation efficace du plateau bit à bit ;
- une profondeur adaptative (Iterative Deepening)³ ;
- un ordonnancement des coups (Move Ordering)⁴ ;
- une fonction d'évaluation heuristique ;
- une optimisation automatique des poids par auto-jeu.

Dans la section suivante, je détaille la méthodologie mise en œuvre et les choix techniques associés.

4 Méthodologie

4.1 Algorithme de recherche : MinMax

Comme mentionné précédemment, j'ai choisi d'utiliser l'algorithme MinMax comme cœur du moteur décisionnel.

Le principe de MinMax est le suivant : à partir de l'état courant du plateau, l'algorithme explore les coups possibles et construit un arbre de recherche. Chaque niveau de l'arbre correspond à un joueur. Lorsque c'est au tour de mon bot, le nœud est dit *MAX* : l'objectif est de maximiser la valeur de la position. Lorsque c'est au tour de l'adversaire, le nœud est dit *MIN* : l'objectif est de minimiser cette valeur.

Autrement dit, l'algorithme simule une alternance de décisions optimales. Le bot choisit le coup qui maximise son gain, en supposant que l'adversaire répondra toujours de la manière la plus défavorable possible. Cette approche permet d'anticiper les conséquences futures d'un coup et d'éviter les décisions trop optimistes.

Formellement, la fonction MinMax peut s'écrire :

$$\text{MinMax}(s) = \begin{cases} \max_{a \in A(s)} \text{MinMax}(s') & \text{si } s \text{ est un nœud MAX} \\ \min_{a \in A(s)} \text{MinMax}(s') & \text{si } s \text{ est un nœud MIN} \end{cases}$$

où $A(s)$ représente l'ensemble des coups possibles depuis l'état s , et s' l'état obtenu après application d'un coup.

Lorsque la profondeur maximale est atteinte ou qu'un état terminal est rencontré, l'exploration s'arrête et une valeur est renvoyée. Cette valeur provient soit du résultat final de la partie, soit d'une fonction d'évaluation heuristique lorsque la recherche est interrompue avant la fin du jeu.

4.2 Représentation du plateau

Au cours du développement de l'algorithme et des premiers tests, j'ai constaté que les performances pouvaient être significativement impactées par la classe `Board` fournie par le projet. En particulier, l'algorithme MinMax implique un grand nombre de copies d'états du plateau lors

2. Technique permettant de réduire le nombre de nœuds explorés en éliminant des branches.

3. Recherche itérative augmentant progressivement la profondeur maximale.

4. Stratégie consistant à explorer en priorité les coups jugés prometteurs afin d'optimiser l'efficacité de l'élagage.

de l'exploration de l'arbre de recherche. Or, la copie répétée d'objets complexes peut devenir coûteuse en temps d'exécution ($\Rightarrow$ moins de nœuds explorés).

Dans une recherche en profondeur, chaque nœud correspond à une nouvelle configuration du jeu. Ainsi, pour une profondeur d et un facteur de branchement⁵ moyen b , le nombre d'états manipulés est de l'ordre de $O(b^d)$. L'efficacité de la représentation des états devient donc un facteur déterminant pour les performances globales.

Afin de réduire ce coût, j'ai décidé d'implémenter mon propre moteur de jeu sous la forme d'une représentation bit à bit du plateau, via la classe `BitBoard`. L'idée consiste à encoder l'ensemble des informations du jeu (nombre de graines, scores, joueur courant, coups valides) dans des variables binaires compactes, manipulées à l'aide d'opérations arithmétiques et logiques rapides.

Schématiquement, le plateau peut être vu comme une structure binaire où chaque trou est représenté par un ensemble de bits :

$$\text{Plateau} = (h_0, h_1, \dots, h_{11}, \text{score}_1, \text{score}_2)$$

Chaque h_i étant encodé sur un nombre fixe de bits, l'ensemble tient dans quelques entiers longs (`long`), ce qui permet :

- une copie très rapide (simple duplication de variables primitives) ;
- une réduction de l'usage mémoire ;
- une simulation plus efficace des coups.

Cette approche est d'ailleurs couramment adoptée dans les moteurs de jeux combinatoires (comme les échecs) et a également été utilisée lors des éditions précédentes du projet, notamment dans le rapport *J'ai Awalé de Travers* de Mathis Saillot (2019–2020), qui décrit une représentation du plateau similaire [3].

Après l'intégration de cette représentation, les performances observées ont été améliorées. À profondeur égale, mon implémentation MinMax obtenait des résultats généralement équivalents ou légèrement supérieurs au bot de démonstration (scores typiques de 2-1 ou 1.5-1.5).

4.3 Optimisation de la recherche

4.3.1 Élagage Alpha-Beta

Afin de réduire le nombre de nœuds explorés lors de la recherche MinMax, j'ai intégré un mécanisme d'élagage Alpha-Beta. Cette optimisation permet de limiter l'exploration de certaines branches de l'arbre lorsque leur résultat ne peut plus influencer la décision finale.

Le principe repose sur le maintien de deux bornes : α , représentant la meilleure valeur déjà trouvée pour le joueur MAX, et β , représentant la meilleure valeur pour le joueur MIN. Lorsqu'il apparaît qu'une branche ne peut pas produire un résultat meilleur que ces bornes, son exploitation est interrompue.

Cette technique ne modifie pas le résultat théorique de l'algorithme MinMax, mais elle permet de réduire significativement le nombre de positions évaluées. Dans le meilleur des cas, la complexité passe de $O(b^d)$ à environ $O(b^{d/2})$, ce qui autorise une profondeur de recherche plus importante à temps constant.

4.3.2 Table de transposition

En réfléchissant aux performances du MinMax, j'ai constaté qu'un grand nombre de positions étaient recalculées plusieurs fois. En effet, différentes séquences de coups peuvent conduire à une même configuration du plateau. Sans mécanisme de mémorisation, l'algorithme réévalue alors inutilement ces états, ce qui augmente fortement le temps de calcul.

5. Nombre moyen de coups possibles à chaque position.

Pour résoudre ce problème, j'ai cherché une méthode permettant d'éviter ces recalculs. Ma première idée a été de m'inspirer du principe de la programmation dynamique (étudié au cours de la L3) : lorsqu'un sous-problème a déjà été résolu, on mémorise son résultat afin de le réutiliser plutôt que de le recalculer. Dans certains cas classiques, cela se traduit par la construction d'une matrice stockant les solutions intermédiaires.

Le principe appliqué ici est similaire et correspond aux tables de transposition utilisées dans les moteurs de jeux combinatoires [4] : associer à chaque position déjà explorée une valeur calculée, afin de la réutiliser si la même position réapparaît dans l'arbre de recherche.

Formellement, on peut représenter la table de transposition comme une structure de la forme :

$$TT : \text{clé}(\text{position}) \longrightarrow (valeur, profondeur, type)$$

Lors de l'exploration d'un nœud, l'algorithme commence par vérifier si la position est déjà présente dans la table. Si c'est le cas, et si la profondeur enregistrée est suffisante, la valeur stockée peut être directement réutilisée, voire provoquer une coupure Alpha-Beta.

Schématiquement, la table peut être vue comme un tableau de la forme :

Clé	Valeur	Profondeur	Type
k_1	v_1	d_1	<i>EXACT</i>
k_2	v_2	d_2	<i>LOWER</i>
k_3	v_3	d_3	<i>UPPER</i>

TABLE 1 – Structure d'une entrée de la table de transposition

Dans mon implémentation, chaque position est encodée via un hachage calculé à partir de la représentation bit à bit du plateau, suivant un principe analogue aux méthodes de hachage utilisées dans les moteurs d'échecs [5, 6]. La table⁶ a une taille fixe de 2^{21} entrées (soit un peu plus de deux millions de positions) et chaque entrée est stockée de manière compacte sur 32 bits, regroupant la valeur évaluée, la profondeur, le type de borne et le meilleur coup associé. Cette structure permet une consultation et une mise à jour très rapides.

L'intégration de cette table de transposition a permis de réduire significativement le nombre de nœuds explorés, améliorant ainsi la profondeur atteinte à temps constant. Elle constitue l'optimisation la plus déterminante du moteur de recherche.

4.3.3 Ordonnancement des coups (Move Ordering)

Afin d'améliorer l'efficacité de l'élagage Alpha-Beta, j'ai mis en place un mécanisme d'ordonnancement des coups. L'idée consiste à explorer en priorité les coups jugés les plus prometteurs.

En effet, plus un bon coup est exploré tôt, plus les bornes α et β sont rapidement resserrées, ce qui augmente les probabilités de provoquer des coupures dans les branches suivantes.

L'ordonnancement repose notamment sur :

- le coup suggéré par la table de transposition ;
- les coups provoquant une capture ;
- certaines heuristiques spécifiques au jeu ;
- d'autres mécanismes (catégories, pv-first, etc.).

Cette stratégie ne modifie pas le résultat de MinMax, mais elle améliore significativement ses performances en pratique.

6. Elle est réinitialisée entre chaque match afin d'éviter toute réutilisation de transpositions issues d'une partie précédente.

4.3.4 Réduction sélective des coups (Late Move Reductions)

L'une des dernières optimisations que j'ai ajoutées à la recherche est l'utilisation des *Late Move Reductions* (LMR) [7]. Le principe consiste à réduire légèrement la profondeur d'exploration pour les coups examinés tardivement dans l'ordre de recherche, car ils ont statistiquement moins de chances d'être les meilleurs.

Cette technique repose sur la qualité de l'ordonnancement décrit précédemment, notamment grâce au coup issu de la table de transposition. Lorsque cet ordre est pertinent, la réduction de profondeur permet de diminuer fortement le nombre de nœuds explorés sans dégrader la qualité de la décision finale.

4.3.5 Profondeur adaptative (Iterative Deepening)

La contrainte temporelle du projet impose de prendre une décision en moins de 100 ms. Il est donc impossible de fixer une profondeur trop élevée sans risquer de dépasser le temps autorisé.

Pour répondre à cette contrainte, j'ai mis en place une recherche par approfondissement itératif (*Iterative Deepening*). Le principe consiste à lancer successivement plusieurs recherches MinMax en augmentant progressivement la profondeur maximale : profondeur 1, puis 2, puis 3, etc., tant que le temps disponible le permet.

À tout instant, la meilleure décision trouvée à la dernière profondeur complète est conservée. Ainsi, même si la recherche est interrompue par la limite de temps, une solution est toujours disponible.

Cette approche permet d'adapter dynamiquement la profondeur atteinte aux ressources disponibles, tout en conservant la qualité de l'algorithme MinMax.

4.4 Fonction d'évaluation

La fonction d'évaluation intervient lorsque la recherche MinMax atteint la profondeur maximale fixée sans parvenir à un état terminal. Dans ce cas, il est nécessaire d'estimer la qualité de la position courante afin de propager une valeur vers les niveaux supérieurs de l'arbre.

Cette fonction doit être suffisamment pertinente pour refléter la situation stratégique, mais également très rapide à calculer. En effet, elle est appelée à chaque nœud feuille exploré : un calcul trop coûteux réduirait le nombre total de positions analysées dans la limite des 100 ms disponibles.

J'ai choisi de construire la fonction d'évaluation comme une combinaison linéaire de six heuristiques :

$$\text{Eval}(s) = \sum_{i=1}^6 w_i \cdot (h_i(s) - h_i(s_{adv}))$$

où h_i représente une heuristique donnée, et w_i son poids associé.

Les heuristiques retenues sont les suivantes :

Heuristique	Principe	Complexité
ScoreDifference	Différence de graines capturées	$O(1)$
Mobility	Différence du nombre de coups légaux	$O(6)$
CaptureValue	Somme des graines capturables	$O(6)$
FamineSafety	Coups adverses provoquant la famine	$O(6)$
EndgameProximity	Urgence de terminer la partie	$O(1)$
ExtraMovesPotential	Coups donnant un tour supplémentaire	$O(6)$

TABLE 2 – Description des heuristiques utilisées dans la fonction d'évaluation

Chaque heuristique est calculée en temps constant ou linéaire par rapport au nombre de trous, ce qui garantit un coût très faible par évaluation.

Comme aux échecs, la stratégie adoptée en début de partie diffère de celle d'une finale. En ouverture et milieu de partie, la mobilité et le contrôle sont importants, tandis qu'en fin de partie, le score direct devient déterminant. J'ai donc choisi de distinguer deux phases de jeu : *early/mid-game* et *late-game*, en doublant les poids associés à chaque heuristique.

La phase est déterminée à partir du nombre total de graines restantes sur le plateau. Soit S le nombre total de graines restantes et $S_{\max}$ le nombre initial (48 graines). On définit alors :

$$\lambda = \frac{S}{S_{\max}}$$

Le coefficient $\lambda \in [0, 1]$ permet d'interpoler entre les deux jeux de poids :

$$w_i(\lambda) = \lambda \cdot w_i^{\text{early}} + (1 - \lambda) \cdot w_i^{\text{late}}$$

Ainsi, lorsque le nombre de graines est élevé ($\lambda \approx 1$), les poids *early* dominent ; lorsqu'il diminue ($\lambda \rightarrow 0$), les poids *late* prennent progressivement plus d'importance. Cette transition continue permet d'adapter dynamiquement le style de jeu au cours de la partie.

4.5 Optimisation des paramètres

L'optimisation des poids de la fonction d'évaluation constitue un élément central du projet. Le réglage manuel de six heuristiques (et douze poids en tenant compte des phases de jeu) s'est rapidement révélé difficile et peu robuste. J'ai donc choisi d'adopter une approche d'optimisation automatique.

Dans un premier temps, je me suis orienté vers deux algorithmes avec lesquels j'avais déjà travaillé par le passé : les algorithmes génétiques et la méthode SPSA. Après analyse, j'ai finalement retenu une troisième approche, CMA-ES, plus adaptée (mais plus difficile également) à la nature du problème.

4.5.1 Algorithme génétique (GA)

Les algorithmes génétiques reposent sur le principe de sélection naturelle [8]. Une population d'individus (ici, les poids) évolue au fil des générations. À chaque itération :

- les individus sont évalués via une fonction de fitness⁷ ;
- les meilleurs sont sélectionnés ;
- des opérations de croisement et de mutation génèrent une nouvelle population.

Cette approche est robuste et relativement simple à mettre en œuvre. Cependant, elle nécessite un grand nombre d'évaluations pour converger. Or, chaque évaluation implique plusieurs parties d'Awélé jouées par le bot, ce qui est coûteux en temps. Compte tenu de la contrainte d'une heure maximale d'entraînement (cf. section 1), cette méthode risquait de ne pas converger vers une solution satisfaisante dans le temps imparti.

4.5.2 Simultaneous Perturbation Stochastic Approximation (SPSA)

La méthode SPSA est une technique d'optimisation stochastique qui approxime le gradient⁸ de la fonction objectif en perturbant simultanément tous les paramètres selon une direction aléatoire [9, 10].

À chaque itération :

- deux évaluations sont réalisées avec des perturbations opposées ;
- une estimation du gradient est calculée ;

7. Fonction qui mesure la qualité d'une solution afin de guider l'optimisation.

8. Vecteur des dérivées partielles indiquant la direction de variation la plus rapide de la fonction.

— les paramètres sont mis à jour dans la direction estimée.

L'un des avantages de SPSA est son efficacité lorsque le calcul exact du gradient est impossible. Cette méthode est notamment utilisée pour l'optimisation de moteurs de jeux comme les moteurs d'échecs [11]. Toutefois, cette méthode effectue une exploration locale autour d'un point initial. Elle nécessite donc de disposer de valeurs de départ déjà très pertinentes. Cette dépendance à une bonne initialisation a donc constitué une limite importante.

4.5.3 Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

Choix de l'algorithme Au vu de la contrainte d'une heure maximale d'entraînement, mon idée initiale était de combiner les deux approches précédentes. Je souhaitais utiliser un algorithme génétique pour explorer globalement l'espace des poids et identifier une région prometteuse, puis affiner localement la solution à l'aide de SPSA. Cette approche hybride (GA + SPSA) [12], m'aurait permis de répartir le temps d'entraînement entre exploration globale et optimisation locale.

Cependant, n'étant pas certain de l'efficacité d'un tel schéma hybride dans le temps imparti, et craignant une complexité de réglage trop importante (répartition du temps, paramétrage des deux méthodes, etc.), je me suis orienté vers la recherche d'un algorithme capable d'assurer à la fois une exploration globale et adaptation locale.

C'est dans ce contexte que je me suis intéressé à l'algorithme *Covariance Matrix Adaptation Evolution Strategy* (CMA-ES), largement utilisé pour l'optimisation continue en présence de fonctions non linéaires, bruitées ou non différentiables [13].

Afin de l'implémenter, j'ai d'abord cherché à comprendre son fonctionnement général ainsi que le rôle de ses principaux paramètres. Je me suis appuyé notamment sur la description algorithmique disponible sur Wikipédia [14] pour structurer mon implémentation. Je ne prétends pas maîtriser l'intégralité des fondements mathématiques sous-jacents (notamment les aspects liés à l'adaptation de la matrice de covariance), mais je pense en avoir compris le principe algorithmique global et la logique d'adaptation des paramètres.

Principe général CMA-ES est un algorithme d'optimisation évolutionnaire adapté aux espaces continus⁹. Contrairement aux algorithmes génétiques classiques, il ne repose pas uniquement sur des mutations aléatoires indépendantes, mais sur une modélisation probabiliste des solutions.

L'idée centrale est de représenter la population non pas comme un ensemble arbitraire d'individus, mais comme un échantillonnage d'une loi normale multivariée :

$$\mathcal{N}(m, \sigma^2 C)$$

où :

- m est le vecteur moyen (estimation actuelle des meilleurs poids) ;
- σ est le pas d'exploration global ;
- C est la matrice de covariance, décrivant les corrélations entre paramètres.

L'algorithme adapte progressivement m , σ et C pour concentrer la recherche vers les régions prometteuses de l'espace des solutions.

Fonctionnement de l'algorithme À chaque génération, l'algorithme procède en trois étapes principales :

1. **Exploration** : génération de λ candidats autour de la moyenne courante à l'aide d'une loi gaussienne :

$$x_k = m + \sigma \cdot \mathcal{N}(0, C)$$

9. Espace où les paramètres peuvent varier de manière continue, et non par valeurs discrètes.

où σ contrôle l'amplitude de la recherche et C sa forme (directions privilégiées).

2. **Évaluation et sélection** : chaque candidat est évalué via la fonction de fitness, puis les μ meilleurs individus sont retenus.
3. **Adaptation** : mise à jour de la moyenne

$$m \leftarrow \sum_{i=1}^{\mu} w_i x_i$$

puis ajustement de la matrice de covariance C et du pas global σ afin d'orienter progressivement la recherche vers les zones prometteuses.

Ainsi, l'algorithme commence par tester un grand nombre de solutions différentes afin d'explorer largement l'espace de recherche. Puis, à mesure que de bonnes solutions sont identifiées, il concentre progressivement la recherche autour de celles-ci pour les améliorer (voir annexe A pour l'algorithme).

Application au projet Dans le cadre de mon projet, chaque individu généré par CMA-ES correspond à un vecteur complet de poids de la fonction d'évaluation (incluant les poids *early* et *late*).

La qualité d'un individu est mesurée à l'aide d'une fonction de fitness basée sur des parties jouées automatiquement. Pour chaque vecteur de poids candidat, le bot affronte plusieurs profils adverses à profondeur fixée. La fitness correspond à une combinaison du résultat des parties (victoire, défaite, égalité) et de l'écart de score obtenu.

Ainsi, CMA-ES ne cherche pas à optimiser une formule théorique, mais les performances réelles du bot en jeu. Il ajuste progressivement les poids pour conserver les configurations qui gagnent le plus de matchs.

4.6 Apprentissage du bot

L'apprentissage repose sur un mécanisme d'auto-jeu et de confrontation à différents profils adverses. Pour chaque individu généré par CMA-ES, plusieurs parties sont jouées contre des bots présentant des styles variés (agressif, défensif, équilibré, auto-jeu, etc.).

La fitness d'un individu est calculée selon la formule suivante :

$$\text{Fitness} = \sum_{p \in \text{Profils}} \alpha_p \cdot (V_p + \beta \cdot \Delta \text{Score}_p)$$

où :

- V_p représente le résultat de la partie (1 victoire, 0.5 nul, 0 défaite) ;
- ΔScore_p est la différence de graines ;
- α_p est un coefficient associé au profil ;
- β pondère l'importance de l'écart de score.

J'ai également intégré un mécanisme d'*Adaptive Opponent Selection* : si un individu obtient de bons résultats contre un profil donné, la probabilité d'affronter des profils plus forts augmente progressivement. À l'inverse, les profils les plus faibles sont moins sélectionnés.

Un bonus spécifique est attribué lorsqu'un individu bat le profil le plus fort (profondeur 7+), afin d'encourager l'optimisation vers un niveau de jeu élevé. Ce bonus renforce l'importance stratégique de ces confrontations dans le calcul de la fitness.

Chaque profil correspond à un vecteur de poids fixe appliqué à la fonction d'évaluation.

5 Résultats

5.1 Benchmark de la représentation du plateau

Afin d'évaluer l'intérêt de la représentation bit à bit, j'ai réalisé un benchmark comparant le plateau *classique* (Board) et le *bitboard* (BitBoard). Le test consiste à effectuer 100 000 000 opérations et à mesurer le temps moyen (en ns/op) de deux actions :

(i) `clone()` ; (ii) `playMove()` exécuté sur un clone (le temps du `clone` n'est pas inclus dans la mesure de `playMove`).

Représentation	clone (ns/op)	playMove (ns/op)
Classique	77.4665	37.0667
Bitboard	11.6246	32.7331
Amélioration (Bitboard)	+85.0%	+11.7%

TABLE 3 – Temps moyen sur 100 000 000 opérations (moyenne en ns/op) et gain du bitboard par rapport à la version classique.

On observe un gain très important sur `clone()`, environ $\times 6.7$ plus rapide en bitboard. Sur `playMove()`, le gain est plus faible mais reste positif. Comme l'algorithme de recherche effectue un grand nombre de copies de positions, cette optimisation a eu un impact direct sur les performances globales du bot.

5.2 Temps de décision en fonction de la profondeur

Afin d'évaluer les performances de mon implémentation du MinMax optimisé, j'ai mesuré le temps nécessaire pour choisir un coup en fonction de la profondeur. Les mesures ont été réalisées sur 30 positions différentes, avec 3 répétitions par position (soit 90 mesures par profondeur).

Le tableau 4 présente une comparaison pour certaines profondeurs représentatives (voir annexe B pour le tableau complet).

Profondeur	MinMax classique (ms)	MinMax optimisé (ms)
6	36.567	1.776
7	152.436	3.898
12	–	145.872
18	–	10664.822

TABLE 4 – Temps moyen (ms) pour choisir un coup selon la profondeur.

On constate que l'implémentation classique devient rapidement impraticable : dès la profondeur 7, le temps dépasse largement la contrainte des 100 ms. En revanche, la version optimisée permet d'atteindre des profondeurs bien plus élevées dans des temps raisonnables.

Il est important de noter qu'il s'agit ici d'un temps moyen mesuré sur un ensemble de positions fixes. En situation réelle de partie, la profondeur atteinte peut être plus élevée ou plus faible selon la complexité de la position. Certaines positions présentent peu de coups légaux ou conduisent à des coupures rapides (alpha-beta), ce qui permet d'explorer plus profondément dans le temps imparti.

5.3 Optimisation des poids par apprentissage

Les poids de la fonction d'évaluation ont été optimisés à l'aide de CMA-ES sur une durée maximale d'une heure. À chaque itération, je sauvegarde notamment le score obtenu (voir annexe C.2), la valeur du meilleur individu, ainsi que les paramètres internes de l'algorithme (pas σ , condition number, moyenne et écart-type des poids).

J'ai pu observer une progression globale croissante du *best_score*.

Afin d'analyser le comportement de l'algorithme, je conserve également l'évolution des poids et de leurs paramètres internes. Il est ainsi possible de projeter deux dimensions des poids et de visualiser la distribution des échantillons générés par CMA-ES à une génération donnée (voir annexe C.1).

6 Conclusion

Au début du projet, j'ai mis du temps à me lancer car je ne suis pas particulièrement attiré par l'intelligence artificielle. En m'intéressant au sujet, j'ai cependant retrouvé des similitudes avec les échecs, un jeu que j'apprécie d'un point de vue informatique, ce qui m'a motivé.

Ce projet m'a permis de réappliquer des concepts vus en licence et sur mon temps libre. Les contraintes de temps et de mémoire ont rendu l'optimisation très stimulante, notamment sur des sujets comme la programmation bit à bit. La découverte et l'implémentation de CMA-ES ont constitué la partie la plus difficile, mais aussi la plus stimulante du projet, même si j'obtenais déjà des résultats plus ou moins satisfaisants avec un algorithme génétique.

Enfin, l'aspect compétitif du tournoi a été un véritable moteur.

Je termine ce projet avec un bot que je trouve satisfaisant, même si je pense encore à certaines améliorations (voir annexe D).

Références

- [1] Wikipedia. *Oware*. <https://en.wikipedia.org/wiki/Oware>
- [2] C. Browne, E. Powley, D. Whitehouse, S. Lucas, P. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. *A Survey of Monte Carlo Tree Search Methods*. IEEE Transactions on Computational Intelligence and AI in Games, 4(1) :1–43, 2012.
- [3] Mathis Saillot. *J'ai Awalé de Travers*. Projet d'Intelligence Artificielle, Avril 2020.
- [4] Chess Programming Wiki. *Transposition Table*. https://www.chessprogramming.org/Transposition_Table
- [5] A. L. Zobrist. *A New Hashing Method with Application for Game Playing*. Technical Report 88, University of Wisconsin, 1970.
- [6] Chess Programming Wiki. *Zobrist Hashing*. https://www.chessprogramming.org/Zobrist_Hashing
- [7] Chess Programming Wiki. *Late Move Reductions*. https://www.chessprogramming.org/Late_Move_Reductions
- [8] Wikipedia. *Genetic Algorithm*. https://en.wikipedia.org/wiki/Genetic_algorithm
- [9] J. C. Spall. *An Overview of the Simultaneous Perturbation Method for Efficient Optimization*. Johns Hopkins APL Technical Digest, 19(4) :482–492, 1998.
- [10] Wikipedia. *Simultaneous perturbation stochastic approximation*. https://en.wikipedia.org/wiki/Simultaneous_perturbation_stochastic_approximation
- [11] T. Romstad, M. Costalba, J. Kiiski, and G. Linscott. *Stockfish Chess Engine - SPSA Parameter Tuning*. Available online : <https://github.com/official-stockfish/Stockfish/wiki>
- [12] B. R. Sahu and S. K. Mishra. *Hybrid Genetic Algorithm with Stochastic Approximation for Noisy Optimization*. Applied Soft Computing, 10(3) : 000–000, 2010.
- [13] N. Hansen and A. Ostermeier. *Completely Derandomized Self-Adaptation in Evolution Strategies*. Evolutionary Computation, 9(2) :159–195, 2001.
- [14] Wikipedia. *Covariance Matrix Adaptation Evolution Strategy*. <https://en.wikipedia.org/wiki/CMA-ES>
- [15] Wikipedia. *Jacobi eigenvalue algorithm*. https://en.wikipedia.org/wiki/Jacobi_eigenvalue_algorithm

A Pseudo-code de l'algorithme CMA-ES (Wikipédia)

Le pseudo-code suivant correspond à la description algorithmique disponible sur Wikipédia. Il présente la version standard de CMA-ES utilisée pour l'optimisation continue.

Algorithm 1: CMA-ES

Input: Moyenne initiale m , pas initial σ

Output: Moyenne optimisée m ou meilleure solution x_1

Fixer λ

Initialiser $C = I$, $p_\sigma = 0$, $p_c = 0$;

while critère d'arrêt non satisfait **do**

for $i = 1$ **to** λ **do**

$x_i \leftarrow \mathcal{N}(m, \sigma^2 C)$;

$f_i \leftarrow \text{fitness}(x_i)$;

 Trier $x_1, \dots, x_\lambda$ selon les valeurs $f_1, \dots, f_\lambda$;

$m' \leftarrow m$;

$m \leftarrow \text{update_m}(x_1, \dots, x_\lambda)$;

$p_\sigma \leftarrow \text{update_ps}(p_\sigma, \sigma^{-1} C^{-1/2} (m - m'))$;

$p_c \leftarrow \text{update_pc}(p_c, \sigma^{-1} (m - m'), \|p_\sigma\|)$;

$C \leftarrow \text{update_C}(C, p_c, (x_1 - m')/\sigma, \dots, (x_\lambda - m')/\sigma)$;

$\sigma \leftarrow \text{update_sigma}(\sigma, \|p_\sigma\|)$;

return m ou x_1

B Temps de décision selon la profondeur (mesures complètes)

Le tableau suivant présente l'ensemble des mesures réalisées pour le temps de décision (en millisecondes) selon la profondeur et l'algorithme utilisé. Chaque valeur correspond à une moyenne sur 90 mesures (30 positions, 3 répétitions).

Algo	Depth	n	Avg (ms)	Median (ms)	p95 (ms)
classic	1	90	0.012	0.012	0.017
classic	2	90	0.048	0.041	0.081
classic	3	90	0.310	0.343	0.509
classic	4	90	1.462	1.441	2.188
classic	5	90	6.693	6.714	9.848
classic	6	90	36.567	33.296	62.506
classic	7	90	152.436	154.470	243.017
classic	8	90	781.656	754.675	1321.780
bitboard	1	90	0.020	0.017	0.042
bitboard	2	90	0.048	0.040	0.094
bitboard	3	90	0.156	0.140	0.273
bitboard	4	90	0.371	0.326	0.671
bitboard	5	90	0.982	0.875	1.612
bitboard	6	90	1.776	1.668	2.969
bitboard	7	90	3.898	3.708	7.628
bitboard	8	90	7.496	7.142	13.903
bitboard	9	90	17.329	17.091	29.574
bitboard	10	90	35.213	35.010	63.106
bitboard	11	90	77.736	74.933	128.539
bitboard	12	90	145.872	136.297	291.293
bitboard	13	90	318.597	300.477	640.685
bitboard	14	90	613.043	594.027	1170.824
bitboard	15	90	1344.660	1208.384	2789.339
bitboard	16	90	2529.170	2309.876	5437.568
bitboard	17	90	5446.411	5135.563	11737.047
bitboard	18	90	10664.822	9506.524	25404.182

TABLE 5 – Temps de décision complet selon la profondeur (moyenne sur 90 mesures)

C Visualisation de l'évolution des poids

C.1 Évolution de la distribution CMA-ES

Afin d'illustrer le comportement de CMA-ES pendant l'apprentissage, j'ai représenté l'évolution des poids au fil des générations.

Chaque figure montre les candidats générés à une génération donnée, projetés sur deux dimensions. On observe le passage progressif :

- d'une phase d'exploration large ;
- à une phase de concentration ;
- puis à un regroupement autour d'une solution optimale.

Les figures suivantes présentent le début et la fin de l'apprentissage (il s'agit simplement d'un exemple, ce n'est pas la configuration finale retenue).

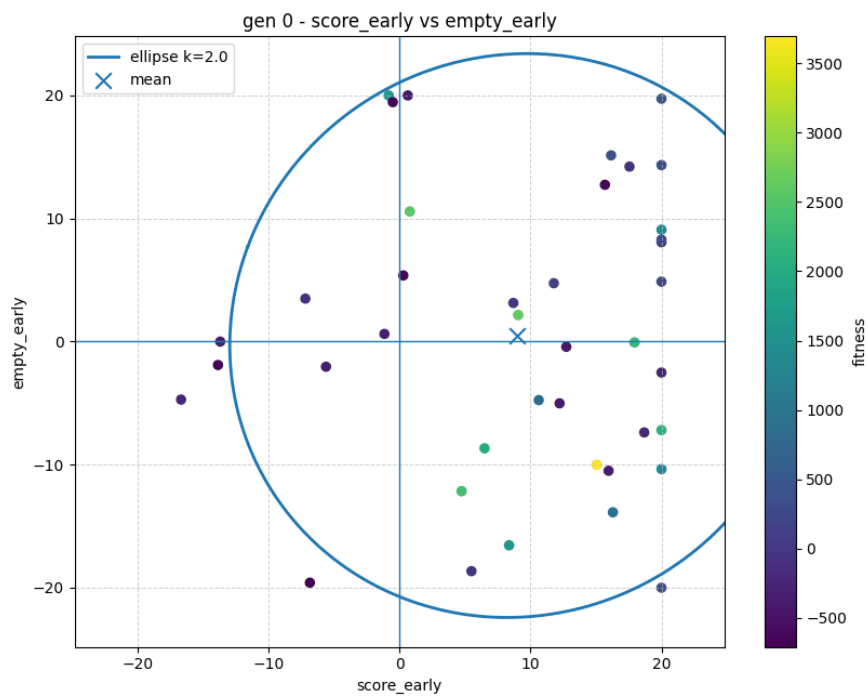


FIGURE 1 – Début de l'apprentissage : exploration large de l'espace des poids.

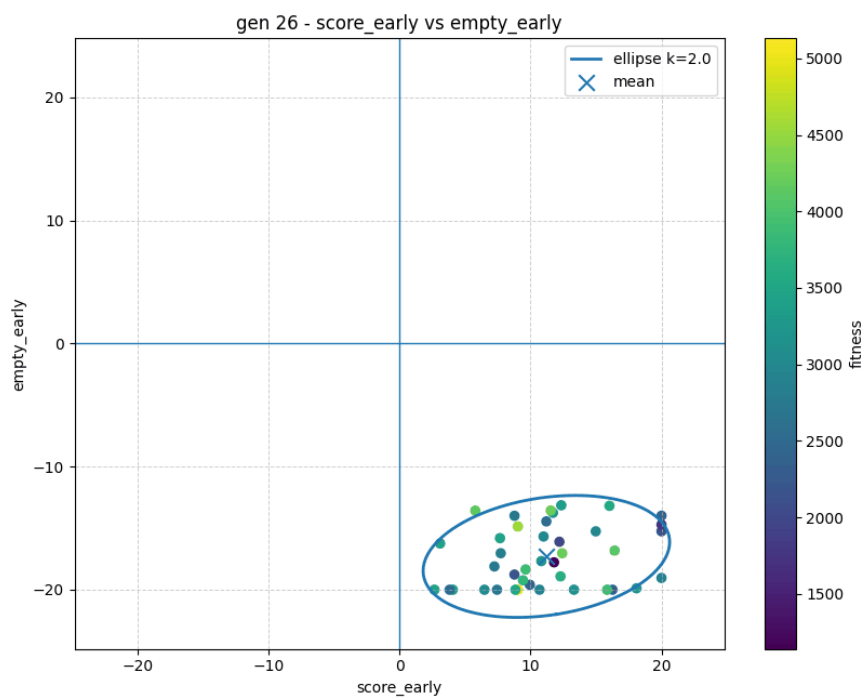


FIGURE 2 – Fin de l'apprentissage : concentration des candidats autour de la moyenne optimisée.

C.2 Évolution des poids (early / late)

La fonction d'évaluation utilise deux jeux de poids : un pour le début/milieu de partie (early) et un pour la fin de partie (late). Pendant l'apprentissage, les meilleurs poids sont sauvegardés à chaque itération.

À partir de ces données, je génère trois visualisations pour mieux analyser les valeurs apprises :

- **Comparaison directe (early vs late)** : visualise, pour chaque heuristique, les deux poids et leur écart.
- **Différence $\Delta = w_{late} - w_{early}$** : montre quelles heuristiques sont davantage favorisées en fin ou en début de partie.
- **Interpolation au cours de la partie** : illustre la transition progressive entre les deux jeux de poids selon le coefficient de phase λ .

Les figures suivantes sont présentées ci-dessous. Elles illustrent ces données à titre d'exemple et ne correspondent pas à la configuration finale retenue.

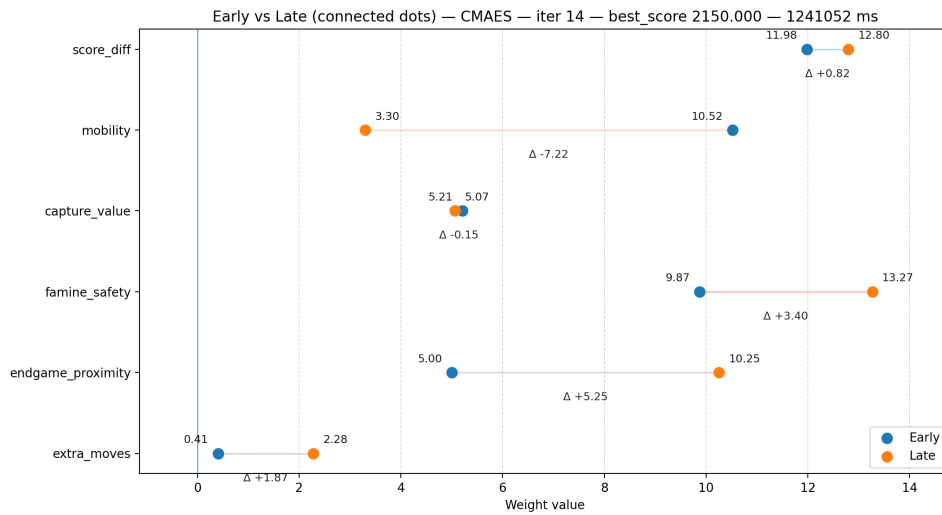


FIGURE 3 – Comparaison des poids early et late (points reliés) pour chaque heuristique.

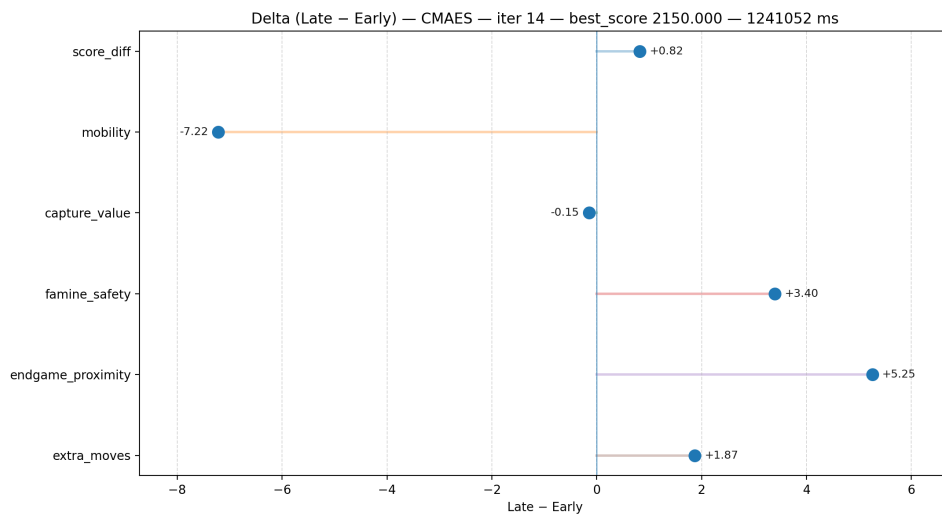


FIGURE 4 – Différence $\Delta = w_{late} - w_{early}$: signe et amplitude de l'écart entre phases.

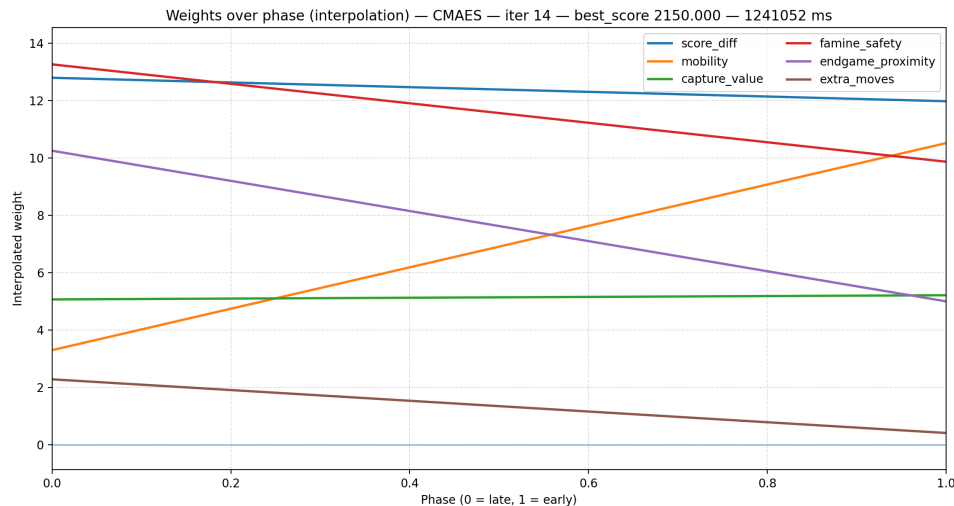


FIGURE 5 – Interpolation des poids en fonction de la phase λ (0 = late, 1 = early).

D Améliorations envisagées

- **History Heuristic** : testée mais non retenue, car elle diminuait légèrement les performances dans mon cas, malgré son usage fréquent dans les moteurs d'échecs.
- **Optimisation mémoire** : la table de transposition n'est jamais remplie à plus de 60%, ce qui représente environ 20 MiB inutilisés ($\Rightarrow$ lui trouver une utilité).
- **Opening book** : idée de précalculer certaines ouvertures, abandonnée faute de temps d'apprentissage suffisant.
- **Base de fins de partie** : création d'un dataset d'endgames pour améliorer les positions finales critiques.
- **Table de transposition persistante** : remplissage d'une TT persistante pendant la phase d'apprentissage.
- **LMR plus avancé** : mise en place d'un système de réduction plus dynamique (type gestion par budget).
- **Heuristiques et poids** : ajout de nouvelles heuristiques et amélioration des poids actuels, probablement encore perfectibles.